MOBILE APP DEVELOPMENT ASSIGNMENT

RICHFIELD
richfield.ac.

Name and Surname: MMANAGA MADIBA

Student ITS No: 402306697

Qualification: BSC IT **Year of Study:** 3rd **Semester:** 2

Assignment due date: 23 October 2026 **Date submitted:** 23 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: M . M _____ **Date:** 2026/10/23 _____

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Table of Contents

Introduction....3
System Design....4
1. Screen flow diagram....4
Database design and data model....4
Screenshots of Every Output....5
Key Code Snippets....8
1. Saving the expiry reminder selection....8
2. Restoring the saved expiry preference....8
3. Creating the expiry options adapter....8
Challenges and Solutions....9
1. Managing pantry data using SQLite....9
2. Saving application settings....9
3. Input validation and reliable records....9
Conclusion....10

Table of Figures

Figure 1: Proposed screen flow for the Smart Pantry Manager....4
Figure 2: Illustrative pantry data model and settings storage....4
Figure 3: The home page....5
Figure 4: Add ingredient screen....5
Figure 5: Ingredient saved successfully....6
Figure 6: Edit ingredient screen....6
Figure 7: Delete ingredient screen....7
Figure 8: Settings Screen....7

Github Link

https://github.com/madibammanaga-del/Smart-Pantry-Manager.git

Introduction

The Smart Pantry Manager is an Android mobile application designed to help users organise food items and ingredients kept in their pantry. Households may find it difficult to remember which ingredients they have, monitor expiry dates, and keep pantry records up to date. These difficulties can contribute to food waste, unnecessary purchases, and poor organisation.

The purpose of the application is to provide a convenient way to add, view, edit, and delete pantry items. The application is developed using Java, while SQLite is used for local storage of pantry records. The settings screen uses Android SharedPreferences to save the user's selected expiry reminder period. Measurement-unit preferences are not included in the current version.

The application is intended for individuals, students, families, and other household users who want to keep track of their food supplies. By maintaining pantry records and recording expiry dates, the application aims to support better household organisation and more responsible food management.

System Design

1. Screen flow diagram

The diagram below presents a proposed flow for the application's main functions. Check that it matches the screens and navigation implemented in the submitted version.

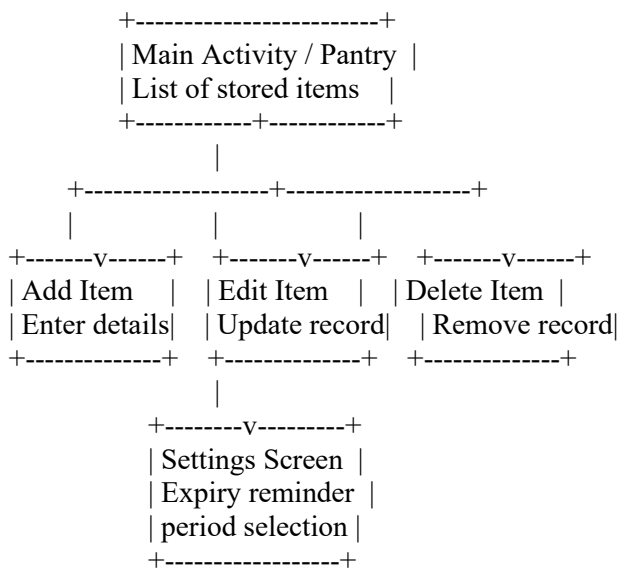
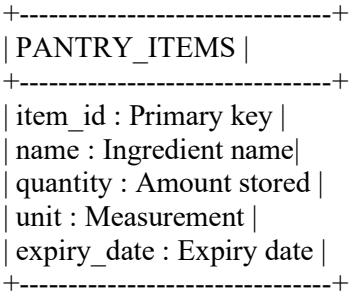


Figure 1: Proposed screen flow for the Smart Pantry Manager.

The main activity acts as the central point for viewing pantry records. Users can add new ingredients and select existing records for editing or deletion. The Settings screen allows users to choose an expiry reminder period. Input validation should be used where implemented to prevent invalid or incomplete records from being saved.

Database design and data model

The application uses SQLite to store pantry information locally. The following is an illustrative data model; the actual table and column names should be checked against DatabaseHelper.java before submission.



Settings stored separately with SharedPreferences:
expiry_reminder_days

Figure 2: Illustrative pantry data model and settings storage.

The pantry table stores each ingredient as a record. The primary key uniquely identifies each record and supports selecting

the correct item for editing or deletion. The name, quantity, unit, and expiry date describe the stored item. The expiry reminder selection is stored separately in SharedPreferences rather than necessarily being part of the SQLite table.

Screenshots of Every Output

Insert genuine screenshots captured from the running Smart Pantry Manager in Android Studio's emulator or on an Android device. Do not use mock-ups, stock images, or screenshots of code as substitutes for application outputs.



Figure 3: The home page

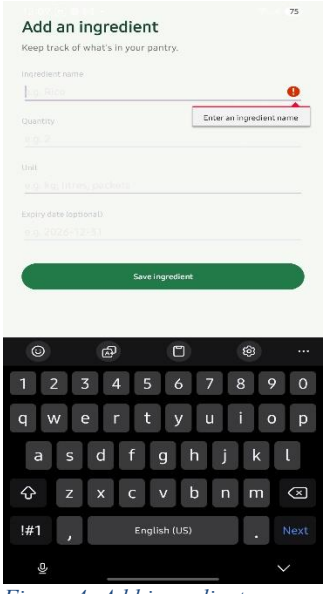


Figure 4: Add ingredient screen

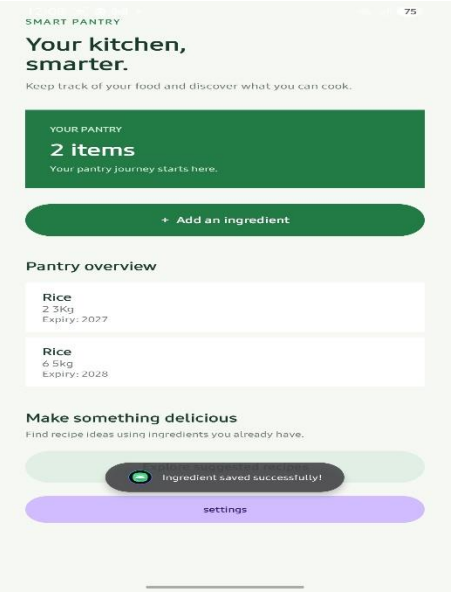


Figure 5: Ingredient saved successfully

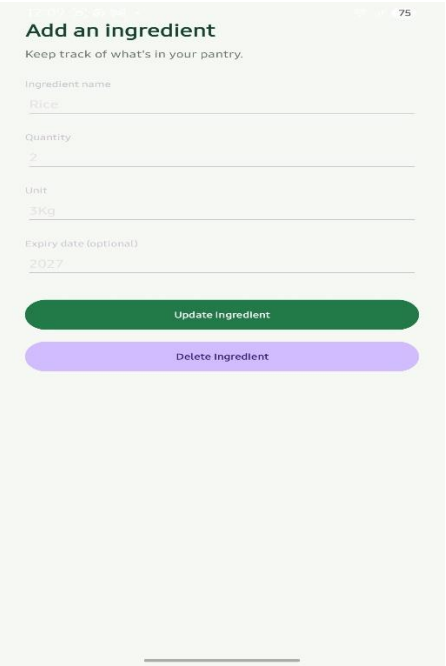


Figure 6: Edit ingredient screen

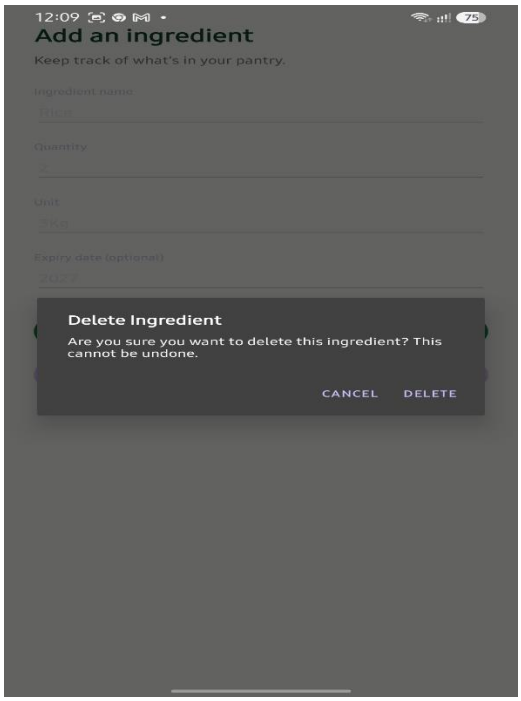


Figure 7: Delete ingredient screen

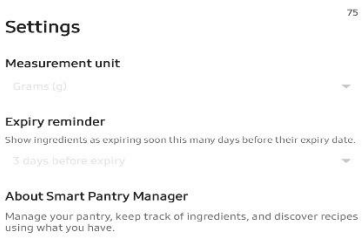


Figure 8: Settings Screen

Key Code Snippets

The following short examples are based on the SettingsActivity.java code. They demonstrate how the expiry reminder option is saved and restored. The final report should include three to five snippets from the actual project.

1. Saving the expiry reminder selection

```
preferences.edit()\n .putInt("expiry_reminder_days", position)\n .apply();
```

This code saves the selected spinner position in SharedPreferences. The preference is stored separately from pantry records, making it possible to restore the user's choice when the settings screen is reopened.

2. Restoring the saved expiry preference

```
int savedExpiryDays = preferences.getInt(\n "expiry_reminder_days",\n 1\n);\nspinnerExpiryDays.setSelection(savedExpiryDays);
```

This code retrieves the saved spinner position and selects it when the Settings screen is created. The default position is 1, which corresponds to the second option, “3 days before expiry,” in the current four-option list. Spinner positions start at zero.

3. Creating the expiry options adapter

```
ArrayAdapter<String> adapter = new ArrayAdapter<>(\n this,\n android.R.layout.simple_spinner_item,\n expiryOptions\n);\nadapter.setDropDownViewResource(\n android.R.layout.simple_spinner_dropdown_item\n);\nspinnerExpiryDays.setAdapter(adapter);
```

The adapter connects the expiry reminder choices to the spinner interface. It allows the available options to be displayed in a drop-down list and lets the user select one. Add one or two further snippets from your own DatabaseHelper.java or validation code if needed to meet the requirement of three to five snippets.

Challenges and Solutions

1. Managing pantry data using SQLite

A potential development challenge is keeping pantry information organised and ensuring that records can be inserted, retrieved, updated, and deleted correctly. SQLite provides local storage for pantry records, while a database helper class can centralise database operations. Describe the exact database issue you encountered and how you tested the solution.

2. Saving application settings

The application needs to retain the selected expiry reminder period after the user leaves the Settings screen. SharedPreferences is used to store the selected spinner position, and the saved value is read again when the screen opens. Testing should confirm that the selected option remains selected after reopening the screen.

3. Input validation and reliable records

Incomplete or invalid ingredient information can reduce the reliability of pantry records. Where input validation is implemented, required fields and numeric quantities should be checked before saving. Error messages should explain what the user needs to correct. Keep this challenge only if it reflects the validation work completed in your app.

Conclusion

The Smart Pantry Manager was developed to address the everyday task of organising pantry ingredients and maintaining useful information about food items. The application brings together a mobile user interface, Java application logic, and local SQLite storage to support the management of pantry records. Its settings screen uses SharedPreferences to retain the user's selected expiry reminder period.

This project provides practical experience in connecting user interface components to application logic and persistent storage. It also demonstrates that different types of data may require different storage approaches: pantry records are stored in SQLite, while the expiry reminder preference is saved separately. Testing add, edit, delete, and list operations is important to confirm that records behave as expected.

A key lesson is that displaying a preference is not the same as implementing every behaviour associated with it. For example, saving an expiry reminder period does not by itself schedule Android notifications. Any notification functionality should be implemented and tested separately before it is described as complete.

Given more time, the application could be improved with search and filtering, clearer expiry status indicators, reliable scheduled notifications, and backup and restore features. Measurement conversion could be considered as a future enhancement, but it is not included in the current version. Any future conversion feature should preserve the original database quantity and unit to avoid changing stored data unexpectedly.

In conclusion, the Smart Pantry Manager demonstrates how mobile application development techniques can be used to support household food organisation. The project has strengthened understanding of Java, Android interface development, SQLite database operations, and persistent settings. Further testing and carefully planned improvements would help make the application more complete, reliable, and convenient for its intended users.

Reference List

Android Developers (n.d.-a) ‘Save key-value data’. Available at: https://developer.android.com/training/data-storage/shared-preferences (Accessed: 23 October 2026).

Android Developers (n.d.-b) ‘Save data using SQLite’. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 23 October 2026).

Android Developers (n.d.-c) ‘Build a responsive UI with ConstraintLayout’. Available at: https://developer.android.com/develop/ui/views/layout/constraint-layout (Accessed: 23 October 2026).

Oracle (n.d.) ‘Java documentation’. Available at: https://docs.oracle.com/en/java/ (Accessed: 23 October 2026).

SQLite (n.d.) ‘SQLite documentation’. Available at: https://www.sqlite.org/docs.html (Accessed: 23 October 2026).